

异步轨迹平衡：解耦探索与学习以实现快速可扩展的大语言模型后训练

Brian Bartoldson¹ Siddharth Venkatraman^{2,3} James Diffenderfer¹ Moksh Jain^{2,3}
Tal Ben-Nun¹ Seanie Lee⁴ Minsu Kim^{2,4} Johan Obando-Ceron^{2,3}
Yoshua Bengio^{2,3,5} Bhavya Kailkhura¹

¹Lawrence Livermore National Laboratory, ²Mila – Quebec AI Institute

³Université de Montréal ⁴KAIST ⁵CIFAR Fellow

{bartoldson,diffenderfer2,kailkhura1}@llnl.gov

{siddarth.venkatraman,moksh.jain}@mila.quebec

摘要

强化学习 (RL) 是大语言模型 (LLM) 后训练的关键组成部分。然而，用于后训练的在线策略算法并不天然地对经验回放缓冲区中多样化的内容具有鲁棒性，而异步离线策略演员可以高效地并行填充这些缓冲区。我们通过异步轨迹平衡 (TBA) 方法，提出了一种针对 LLM 的异步 RL 方法，该方法利用了基于原则的离线策略 TB 目标。在数学推理、偏好调整和自动化红队任务上，我们对从 Pythia 410M 到 Qwen 2.5 7B 的不同模型进行了后训练，发现 TBA 相较于强大的基线如在线 DPO 和 Dr. GRPO 提供了速度和性能上的提升。除了 TBA 的性能优势（即使异步程度增加也能保持高准确率）和加速（加快 4× 倍或更多），我们还展示了其奖励优先和近期优先的采样方法能够在数据生成扩展时进一步带来收益。我们的代码可在 <https://github.com/bbartoldson/TBA> 获取。

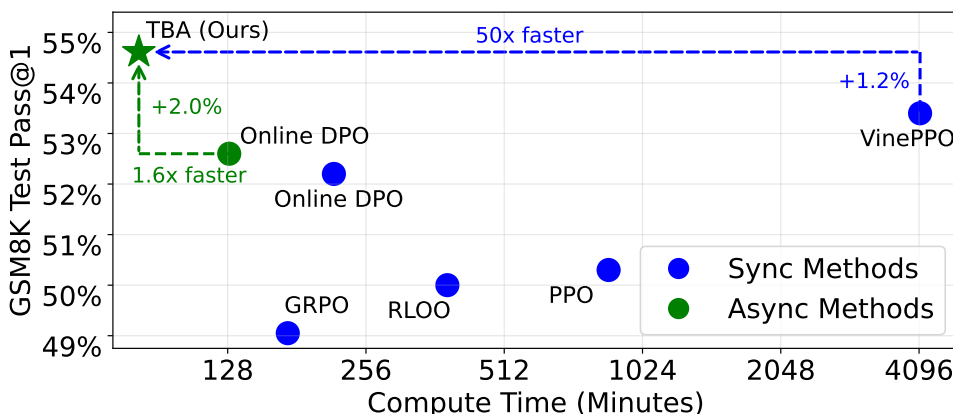


图 1：TBA 在 GSM8K 数学推理任务上表现出色。所有绘制点均使用 4xA100 GPU（或相当的 L40S GPU）。DPO 和 RLOO 基线来自 Noukhovitch 等人的研究 [48]，PPO 和 VinePPO 基线来自 Kazemnejad 等人的研究 [31]。基线模型是经过 SFT 的 RhoMath-1B[37] 模型，在 SFT 后但 RL 前的准确率为 40%。附录 B 有详细信息。

1 引言

通过强化学习 (RL) 进行后训练对于增强大语言模型 (LLM) 至关重要, 使其与人类偏好对齐并提高其推理能力 [9, 29]。然而, 广泛使用的 RL 算法如近端策略优化 (PPO) [59] 和 REINFORCE Leave-One-Out (RLOO) [2] 存在一个基本局限性: 它们是在线策略的, 这意味着数据生成和策略更新是顺序发生的。这种依赖关系造成了瓶颈, 降低了资源利用率。此外, 扩大在线策略数据生成的效益可能有限 [24]。

本文介绍了一种基于异步强化学习的方法——轨迹平衡异步算法 (Trajectory Balance with Asynchrony, 简称 TBA), 旨在高效且可扩展地利用计算资源进行大语言模型的后训练。TBA 采用了基于轨迹平衡 (Trajectory Balance, 简称 TB) 的离策略训练目标 [41], 我们假设这种方法可以改善使用异步强化学习时的数据生成与策略更新之间的解耦, 从而缓解关键的强化学习瓶颈。

在数学推理、偏好调整和自动化红队任务中, 我们发现 TBA 相较于现有的大语言模型后训练方法, 具有三个广泛的优势: 1) 稳定的离策略强化学习, 解锁异步处理以实现大规模并行化和减少实际运行时间——请参见图 1 和图 3。

2) 从多样化的回放缓冲区中采样, 提高探索效率并防止模式崩塌。3) 具备可扩展的搜索兼容性, 有助于稀疏奖励设置下的自动化红队任务。

我们的主要贡献总结如下:

- 我们提出了 TBA, 这是一种用于快速、可扩展的大语言模型后训练的分布式异步强化学习框架。
- 我们展示了使用 TBA 进行后训练的大语言模型能够匹配或超越现有方法的性能, 这说明了 TB 利用离策略数据在异步强化学习中的能力。
- 我们证明了在数学推理、偏好调整和自动化红队任务中, T 的 RL 速度有显著提升 (4 $\times$ 至 50 $\times$)。

通过实现高质量且快速的离策略后训练, TBA 有助于实现大语言模型的可扩展性和有效性对齐, 确保大型模型能够更高效地被优化以适应现实世界的部署。

2 相关工作

强化学习对语言模型的微调 强化学习 (RL) 已成为训练大型语言模型 (LLM) 不可或缺的一部分 [16, 50]。特别是, RL 已经成为使语言模型与人类偏好对齐的默认方法 [9, 88, 64, 51]。许多工作依赖于近端策略优化 (PPO) 算法 [59], 这是一种在策略的 RL 算法, 在不同设置下表现出色, 成为微调 LLM 的默认选择。除了 PPO 之外, 其他在策略目标如 REINFORCE [2] 及其变体 GRPO [61] 和 VinePPO [31] 也在语言模型的背景下进行了研究。与基于 PPO 的微调相对的是拒绝采样微调, 这种方法受到中野等人提出的最佳之 n 推理方法的启发 [47]。董等人、古尔切雷等人以及王等人最近的工作扩展了这一概念, 通过生成 n 候选响应来应对每个提示, 使用学习到的奖励函数对它们进行排序, 并根据排名最高的响应微调模型 [12][17][76]。另一方面, 直接偏好学习方法完全跳过了奖励建模, 训练语言模型直接在偏好模型下优化响应 [55, 3, 67]。胡等人介绍了 GFlowNet 微调, 利用离策略的 GFlowNet 算法进行语言模型的微调, 这是我们在此基础上进一步发展的内容 [25]。

异步分布式强化学习 分布式强化学习将执行者/搜索者、学习者和环境分散到一系列计算资源上。异步分布式强化学习这样做的方式是, 搜索者和训练器进程不一定共享相同的权重, 这可以显著提高训练速度 [46, 77] 并促进复杂高维领域的强化学习 [21, 26, 23]。

在这个领域, 一种基础方法是异步优势演员-评论家 (A3C) 算法 [44]。在 A3C 中, 多个并行工作的单元异步地与环境交互, 并将梯度传递给中央节点。我们的异步分布式强化学习方法更接近于重要性加权演员-学习者架构 (IMPALA) 方法 [13], 该方法将经验轨迹 (状态、动作和奖励元组) 传递给中央节点。与标准的强化学习环境不同, 在标准环境中基于价值的离策略学习目标能够实现原则性的异步训练, 语言模型则提出了独特的挑战, 因为学习价值函数非常困难 [18]。最近的工作也利用这些异步强化学习框架在大语言模型后期训练中对近似于在线数据进行训练 [48, 15]。与这些近期工作相比, 我们的方法使用了一个原则性的离策略学习目标, 允许对任何离策略轨迹进行训练。

自动化红队测试 通过对抗性互动，大语言模型的红队测试可以澄清目标模型的鲁棒性和风险。自动生成这些对抗性场景的自动化红队测试框架可以帮助更快地发现模型中的漏洞、偏差和意外行为，从而在部署前进行预防性缓解措施 [78, 30]。

佩雷斯等人 [53] 提出使用强化学习训练语言模型，以发现引发某些目标大语言模型产生有害响应的提示。然而，标准的强化学习方法容易发生模式崩塌，并且无法实现成功红队测试所必需的攻击多样性。洪等人 [22] 引入了一种好奇心奖励来鼓励生成多样化的红队测试提示，而萨姆维利安等人 [58] 则提出从一个池中抽样攻击提示，并使用辅助大语言模型迭代地变异提示。李等人 [36] 提出使用 GFlowNet 微调后跟随最大似然估计平滑来生成多样化、可转移且有效的提示。我们的红队测试实验通过我们的分布式异步框架增强了他们的 TB 目标优化。

3 预备知识

3.1 KL 正则化的强化学习作为概率推理 我们研究了微调预训练语言模型 π_{ref} 与奖励模型 r_ϕ 的问题。对于数学推理任务 [10]，我们的奖励模型简单地计算响应的正确性，而我们在对齐偏好微调 [88] 和红队测试实验 [53] 中使用的奖励模型则是通过人类偏好数据优化得到的。值得注意的是，具有学习奖励函数的奖励最大化强化学习容易受到奖励函数虚假模式的影响 [63, 52, 14]，这可能导致响应性能不佳且多样性较低。这一问题通过约束微调后的模型在 KL 发散意义上接近初始模型来解决：

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [\mathbb{E}_{\mathbf{y} \sim \pi(\mathbf{y}|\mathbf{x})} [r_\phi(\mathbf{y}; \mathbf{x})] - \beta \mathbb{D}_{\text{KL}}(\pi(\cdot | \mathbf{x}) || \pi_{\text{ref}}(\cdot | \mathbf{x}))]. \quad (1)$$

在线 RL 算法，如 PPO[59, 64] 和 REINFORCE[79, 33, 2] 可用于优化模型（即策略），而离线目标如 DPO[55] 可以直接在偏好数据上训练模型，并渐近地达到相同的最优策略。

式 1 可以从概率的角度解释为一个贝叶斯后验推断问题 [34]。式 1 的最优策略给定为：

$$\pi^*(\mathbf{y} | \mathbf{x}) \propto \pi_{\text{ref}}(\mathbf{y} | \mathbf{x}) \exp(\beta^{-1} r_\phi(\mathbf{y}; \mathbf{x})). \quad (2)$$

诸如吉布斯采样等方法可以在无需微调的情况下生成最优策略的样本 [80]。这些 MCMC 方法在推理时可能非常昂贵，并且对于长序列来说是不可处理的。在概率解释下，最大化公式 1 的在线策略强化学习等同于最小化逆 KL 散度的变分推断，相对于后验密度而言 [34]。然而，逆 KL 散度优化容易发生模式崩塌，并且需要在线策略样本。依赖在线策略样本可能会限制可扩展性，因为我们不能使用可以并行填充的大规模回放缓冲区，因为策略的新更新会使旧的回放缓冲区示例失去在线性质。实际上，这意味着策略可能会陷入次优解并停止探索，或者为了获得高奖励而牺牲多样性。这促使我们考虑一种替代的离线策略变分推断，以便有效地利用可扩展的计算资源进行灵活探索。

3.2 GFlowNets

生成流网络（GFlowNets; 4, 5）是一种用于离策略训练层次生成模型的框架，该模型能够按照给定的未归一化密度（奖励）函数进行采样。GFlowNets 将概率推理表述为一个顺序决策问题，学习一种策略来构建对象（例如 通过组合构建块（例如标记）并优化基于一致性的目标来生成序列。GFlowNet 的目标已被用于微调自回归算法 [25, 36] 以及离散扩散语言模型 [72]。GFlowNets 是最大熵强化学习算法 [87]，值得注意的是，在序列生成问题中等价于路径一致性学习算法 [45][4]。

为了将语言模型微调到从方程 2 中采样，我们可以设置奖励为 $R(\mathbf{y}; \mathbf{x}) = \pi_{\text{ref}}(\mathbf{y} | \mathbf{x}) \exp(\beta^{-1} r_\phi(\mathbf{y}; \mathbf{x}))$ 。遵循 Lee 等人 [36] 的方法，我们使用轨迹平衡目标 [42] 对语言模型策略 π_θ 进行训练，该目标定义在响应 $\mathbf{y}$ 上

$$\mathcal{L}_{\text{TB}}(\mathbf{y}, \mathbf{x}; \theta) = \left(\log \frac{Z(\mathbf{x}) \pi_{\theta}(\mathbf{y} | \mathbf{x})}{R(\mathbf{y}; \mathbf{x})} \right)^2. \quad (3)$$

$Z(\mathbf{x})$ 是查询 $\mathbf{x}$ 的正标量函数，响应 $\mathbf{y}$ 是一系列标记。当 $\mathcal{L}_{\text{TB}}$ 被最小化时， $Z(\mathbf{x})$ 是后验的配分函数（即 $Z(\mathbf{x}) = \sum_{\mathbf{y}} R(\mathbf{y}; \mathbf{x})$ ）。否则它作为控制变量，减少 TB 梯度的方差。我们不训练一个价值网络来估计 $Z(\mathbf{x})$ ，而是使用 VarGrad 版本的轨迹平衡，用一个 K 样本批量估计 [57, 49, 83, 60, 72] 替换学习到的 $Z(\mathbf{x})$ 。响应

Given K $\{\mathbf{y}^{(i,j)}\}$ 对于查询 $j=1$ ，可以计算一批 $\mathbf{x}^{(i)}$ 的估计值 $\hat{Z}(\mathbf{x}^{(i)})$

$$\log \hat{Z}(\mathbf{x}^{(i)}) = \frac{1}{K} \sum_{j=1}^K \left(\log \pi_{\text{ref}}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) + \frac{1}{\beta} r_{\phi}(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)}) \right). \quad (4)$$

(分离的) 估计值 $\hat{Z}$ 可以插入方程 3 中以获得一批 $\mathbf{B} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i,j)})_{j=1}^K\}_{i=1}^B$ 得到

$$\mathcal{L}_{\text{TB}}^{\text{VarGrad}}(\mathbf{B}; \theta) = \frac{1}{BK} \sum_{i=1, j=1}^{i=B, j=K} \left(\text{STOP-GRAD} [\log \hat{Z}(\mathbf{x}^{(i)})] + \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \log \pi_{\text{ref}}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \frac{1}{\beta} r_{\phi}(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)}) \right)^2. \quad (5)$$

轨迹平衡的一个重要性质是它是离策略的。在训练过程中， $\mathbf{y}$ 可以从具有完全支持的任何分布中采样 [4]。这使得可以使用各种探索策略 [56, 32] 以及回放缓冲区 [62, 71]。在微调语言模型的背景下，这一目标的离策略性质使其成为异步训练的自然选择。

3.3 与其他强化学习目标的区别 我们比较了上述 VarGrad TB 变体和 Proximal RLOO 的梯度，后者是 RLOO 的一个离策略变体，其离策略鲁棒性源自一个（裁剪过的）重要性采样 (IS) 比率 [48]。请参见附录 D 以获取更多比较。使用 K 样本，Proximal RLOO 对于样本 $\mathbf{y}^{(j)}$ 的梯度

$$\nabla \mathcal{J}_{\text{P-RLOO}}(\theta) = \frac{\pi_{\theta}(\mathbf{y}^{(j)} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}^{(j)} | \mathbf{x})} \hat{A}(\mathbf{y}^{(j)} | \mathbf{x}) \nabla_{\theta} \log \pi_{\theta}(\mathbf{y}^{(j)} | \mathbf{x}), \quad (6)$$

的优势 $\hat{A}(\mathbf{y}^{(j)} | \mathbf{x}) = \frac{\pi_{\theta}(\mathbf{y}^{(j)} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}^{(j)} | \mathbf{x})} \left(\log \frac{\pi_{\theta}(\mathbf{y}^{(j)} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}^{(j)} | \mathbf{x})} + \frac{1}{\beta} r_{\phi}(\mathbf{y}^{(j)}; \mathbf{x}) \right)$ 为了简洁而省略。使用 TB 目标函数的梯度对于 $\mathbf{y}^{(j)}$ ，对应于等式 5 中的损失，最多乘以一个常数，等于以下内容（推导过程见附录 A）： $\mathcal{J}_{\text{TB}}$

$$\nabla \mathcal{J}_{\text{TB}}(\theta) = \hat{A}(\mathbf{y}^{(j)} | \mathbf{x}) \nabla_{\theta} \log \pi_{\theta}(\mathbf{y}^{(j)} | \mathbf{x}), \quad (7)$$

$$\hat{A}(\mathbf{y}^{(j)} | \mathbf{x}) = [r(\mathbf{y}^{(j)}, \mathbf{x}) - \beta \log \frac{\pi_{\theta}(\mathbf{y}^{(j)} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}^{(j)} | \mathbf{x})}] - \frac{1}{K} \sum_i [r(\mathbf{y}^{(i)}, \mathbf{x}) - \beta \log \frac{\pi_{\theta}(\mathbf{y}^{(i)} | \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}^{(i)} | \mathbf{x})}].$$

其中，对于在线策略数据 $\mathbf{y}$ ， $\text{TB}^{\text{VarGrad}}$ 等价于均值基线 REINFORCE [79] 和 KL 散度正则化的奖励 [88]。请注意，我们在离策略数据上使用 TB，即不从 π_{θ} 中采样，在这种情况下等价关系不再成立。

实践中，我们发现离策略数据上的性能对系数 β 敏感，较大的值倾向于促进稳定性，而较小的值倾向于促进准确性的提高。我们发现可以通过系数退火计划或重置参考策略 [38] 来同时获得这两种好处。值得注意的是，Kimi K1.5 和 K2 通过重置参考策略并使用几乎匹配 TB 的目标进行了 RL 调优 [69, 68]。然而，这些 Kimi 方法偏离了等式 7，排除了控制变量中的平均对数概率比。此外，它们的重置方法导致对数概率比的分母始终使用生成轨迹的行为策略作为参考，而 TBA 可以较少频繁地重置，更长时间地使用原始基础策略作为参考。在第 6 节和附录 D 中，我们将重要性采样引入等式 7 中的 TBA 目标，并展示了它与 CISPO IS 裁剪 [8] 或 IS 比率掩码 [85, 86] 配合良好。

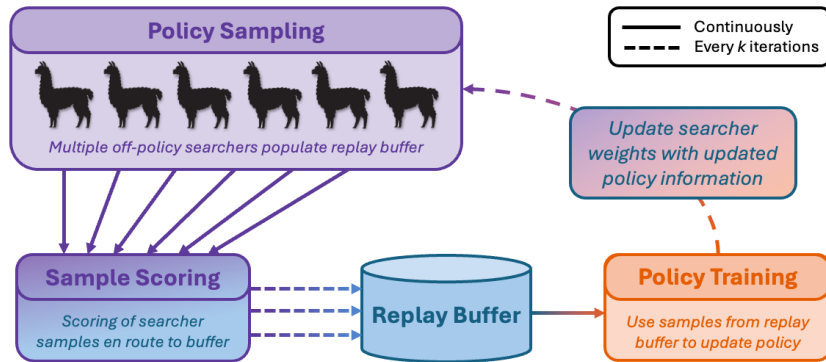


图 2：使用 TBA 进行快速可扩展的大规模语言模型后训练。连续地（实线），多个 SEARCHER 节点（左侧）收集轨迹，而 TRAINER 节点（右侧）从回放缓冲区中采样以离策略训练策略。定期地（虚线），更新后的策略权重被发送到 SEARCHER 节点，新的轨迹被添加到 TRAINER 节点的缓冲区中。这避免了任何给定节点的瓶颈，这些节点可以是一个或多个 GPU，保持资源利用率高。

4 TBA：快速可扩展的大规模语言模型后训练

TBA 将等式 5 中的 TB 梯度整合进一个异步分布式强化学习框架，用于训练后的语言模型。异步强化学习将数据生成与模型更新解耦，从而提高资源利用率并缩短训练时间。同时，离策略 TB 目标促进了从由异步性引起的离策略训练数据中高效学习。

我们测试了两种 TBA 变体。第一种从零开始实现 TBA，通过修改 Hugging Face TRL 库中的 RLOO 训练器类（例如 [74]）来使用公式 5，添加一个调度器用于 β ，集成回放缓冲区和采样方法，并行化训练和搜索以支持异步强化学习——我们将在下面详细讨论这种变体的细节。第二种变体 TBA₊，使用异步 PRIME-RL 代码库 [28]，我们只需添加与公式 7 对应的更新规则和参考策略重置。TBA₊ 比 TBA 具有更少的可调参数，并支持多 GPU 训练过程，使我们能够在更简单的设置下测试 TB 用于 LLM 的异步强化学习，并且可以使用更大的模型/上下文。

虽然 TBA₊ 侧重于易用性和多 GPU 训练，我们的从零开始实现的 TBA 则更注重速度和搜索。例如，确保数据具有恒定水平的离策略性（如同 PRIME-RL 所做的那样）可能会重新引入异步强化学习试图消除的瓶颈，因此 TBA 连续独立地运行训练和搜索过程，仅在每 k 步同步一次（即“同步周期”）。此外，无论其过时程度如何，每次完成的数据都会被存储在 TBA 的回放缓冲区中（例如通过基于奖励的采样）。正如下面讨论的， m 是 TBA 采样相对“最符合策略”的数据的概率；即，在最近一次同步步骤中添加的数据。

图 2 可视化了我们的 TBA 实现。它利用重叠的训练和搜索过程，并使用缓冲区来存储轨迹。特别是，TBA 使用一个 TRAINER 节点和一个或多个 SEARCHER 节点，将离策略数据收集到共享的回放缓冲区中 $\mathcal{D}_{\text{global}}$ 。在这里，节点是指能够执行所有训练或搜索操作的计算资源——它可以是一组多个 GPU，甚至可以是一个 GPU 的一部分。在我们的 TBA 实验中，一个节点始终是一个 GPU：例如，在给定 16 个 GPU 的情况下，我们使用 15 个 SEARCHER 节点和 1 个 TRAINER 节点。

在仅包含两个节点（例如两个 GPU）的集群中，分离 SEARCHER 和 TRAINER 是非常有利的，因为大语言模型的策略展开是昂贵的顺序解码过程，而训练只需要通过单次前向传递对整个序列进行并行似然评估。因此，对于可以容忍离线异步在线训练的目标函数，可以通过在一个节点上持续运行训练而不暂停生成展开来实现巨大的时间加速。在接下来的小节中，我们将提供更多关于从零开始实现 TBA 的细节。

4.1 利用 SEARCHER 节点扩展数据收集 TBA 的 SEARCHER 节点各自携带一个本地延迟副本 $\pi_{\theta'}$ 的 TRAINER 策略 π_{θ} 。为了生成策略回放，从数据集中采样查询 $\mathbf{x}$ ，并由本地策略生成一批 K 响应 $\mathbf{y} \sim \pi_{\theta'}(\mathbf{y}|\mathbf{x})$ ，这些响应随后通过奖励模型 $r_{\phi}(\mathbf{y}; \mathbf{x})$ 进行评估。如同 Nourkhovitch

等人 [48]，我们使用 vLLM [35] 进行更快的生成。这些 $(\mathbf{x}, \mathbf{y}, r_\phi(\mathbf{y}; \mathbf{x}))$ 元组被存储在搜索器的本地回放缓冲区 $\mathcal{D}_{\text{local}}$ 中。我们还添加了训练器在最后一次同步时的步骤数到存储的元组中，这使我们能够了解生成数据的离策略程度——这一信息可以被训练器用于优先从最近的生成数据中进行采样。

每 k 次优化步骤，搜索和训练暂停，将每个 SEARCHER 节点的本地回放缓冲区 $\mathcal{D}_{\text{local}}$ 拉入全局回放缓冲区 $\mathcal{D}_{\text{global}}$ ，并更新搜索器的本地策略为训练器的策略。全局缓冲区维护一个列表，记录所有生成的响应及其对应的奖励值。

异步性的主要动机在于其与展开缩放的兼容性，这有助于增强探索或增加序列长度。例如，对于给定的查询，我们生成 $S > K$ 样本——即使每次查询仅使用 K 样本更新模型——以此来缓解由于 K 独立模型展开不一定会产生 K 独特序列而导致的多样性不足问题。相关地，未来工作可以将简单的离策略推理技术应用于 SEARCHER 节点，如随机采样 Softmax 函数温度，或使用束搜索等替代解码技术。我们预期此类方法特别有助于在稀疏奖励设置中发现解决方案。

4.2 使用 TRAINER 进行异步更新

训练器利用离策略轨迹平衡（公式 5）在全局回放缓冲区上训练策略。我们采样一批 B 查询 $\mathcal{D}_{\text{global}}$ ，每个查询有 K 响应及其对应的奖励：

$$\{\mathbf{x}^{(i)}, \mathbf{y}^{(i,j)}, r_\phi(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)})\}_{i=1, j=1}^{i=B, j=K} \sim \mathcal{D}_{\text{global}}.$$

然后我们在公式 5 中计算损失，并利用它来更新策略。如果给定查询的唯一样本少于 K ，我们将进行有放回抽样。

一个关键的设计选择是从回放缓冲区 $\mathcal{D}_{\text{global}}$ 中采样的策略。最简单的方法是对查询进行均匀抽样，然后对选定查询关联的样本进行均匀抽样，这种方法可能不是最优的，特别是当高回报样本稀疏时。通过倾斜采样分布向高回报序列，奖励优先级可以解决这个问题；然而，仅关注高回报数据可能导致模式崩塌并减少策略多样性。

为了平衡这些考虑，我们交替使用两种采样策略：一种优先考虑近期性——即轨迹是否在最近的同步步骤中添加到缓冲区——另一种优先考虑奖励。在优先考虑奖励时，我们考虑奖励值的 Softmax 函数（以鼓励抽取高回报响应）和均匀分布（以鼓励等概率抽取高低回报响应）。对于每批查询中的每个查询，我们随机切换优先考虑奖励还是近期性，将分配给每种策略的查询比例视为可调超参数 m ，我们在第 5.4 节研究这一点。

5 实验评估

我们评估 TBA 在三种常见的大语言模型后训练强化学习流水线中的有效性。大语言模型后训练强化学习用于增强代理和推理任务的能力，是一个关键但尚不成熟的研究领域，基线方法需要数小时甚至数天的时间。这些传统方法通常依赖于生成成本高昂的在线数据，因此未能有效利用可用的计算资源。值得注意的是，这种低效性在扩展到更大的分布式系统时可能会变得尤为有害，这可能是处理稀疏奖励领域所必需的，这些领域需要增加采样量。总体而言，我们发现 TBA 是一种高性能、快速且可扩展的解决方案。

5.1 任务

- **数学推理 (Mathematical Reasoning (MR))**：我们研究 GSM8K 任务，该任务包含小学水平的数学问题，并基于最终答案的精确匹配给出二元奖励 [10]。我们采用 Kazemnejad 等人 [31] 以及 Noukhovitch 等人 [48] 的设置，使用经过 SFT 训练的 RhoMath-1B[37] 模型作为 RL 后训练的基础模型。

- **偏好微调 (Preference Fine-Tuning (PFT))**：我们考虑使用从人类偏好数据中学习到的奖励函数对语言模型进行微调的任务。具体来说，我们研究 TL;DR 摘要任务，目标是对 reddit 帖子撰写简短摘要 [64]。参考 Noukhovitch 等人 [48]，我们将 Pythia [6] 作为策略和奖励模型的基础模型，并使用 Noukhovitch 等人使用的 SFT 版本 [48]。

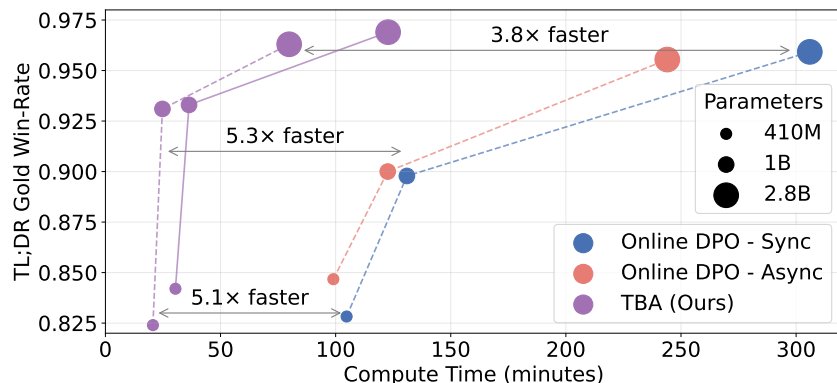


图 3: TBA 扩展搜索并提高 PFT 摘要任务上的 RL 效率。所有绘制的点均使用 4xA100 GPU, 但 TBA 分配 3 个 GPU 用于搜索, 而在线 DPO 分配 1 个 GPU 用于搜索。TBA 生成大规模离线数据, 其轨迹平衡目标可以利用这些数据, 从而产生巨大的效率优势。在线 DPO 基线取自 Noukhovitch 等人 [48]。虚线和实线分别使用 256 和 425 次更新。附录 B 有详细信息。

• **红队测试 (Red Teaming (RT))** : 我们调查自动化红队测试, 这是 LLM 后训练过程中的另一个关键步骤。目标是发现能够引发目标模型有害响应的提示, 通过毒性分类器来衡量。我们遵循 Lee 等人的设置 [36], 应用相同的模型: 我们的小规模实验使用 GPT-2 [54] 作为攻击者模型, GPT-2 (指令调整版) 作为受害者模型, 以及基于 RoBERTa 的毒性分类器 [73]; 我们的大规模实验使用 Llama-3.2-1B [43] 作为攻击者模型, Llama-3.1-8B-Instruct 作为受害者模型, 以及 LlamaGuard-3-8B [43] 来测量毒性, 对受害者模型的多个响应进行平均。

对于所有任务, 我们仅研究基线工作流程中的 RL 后训练组件。具体而言, 我们从经过 SFT 的检查点开始, 然后进行 RL。值得注意的是, Lee 等人 [36] 在 RL 之后还包括一个最大似然估计训练阶段, 该阶段使用 RL 过程中生成的缓冲区, 这可以提升性能, 但在此未进行研究。使用这些基线的代码, 我们遵循其设置和超参数, 偏差详见附录 B。对于 MR 和 PFT, 我们通过增强 Noukhovitch 等人的 RLOO 训练器 [48] 来实现 TBA, 添加了我们的分布式异步 RL 框架和 TB 目标 (方程 5)。对于 RT, 我们通过增强 Lee 等人的轨迹平衡训练器 [36] 来实现 TBA, 添加了我们的分布式异步 RL 框架。

5.2 度量与基线

数学推理 (Mathematical Reasoning (MR)) 度量与基线。我们遵循 Noukhovitch 等人的评估设置 [48], 在 GSM8K 上计算 pass@1 [10] 测试数据集采用贪婪解码。先前的工作 [31, 48] 在 GSM8K 训练集上对 SFT 后的 RhoMath-1B 应用 RL 后训练 [37] 模型最初在测试集上获得 40.3% 的准确率。我们将 TBA 后训练方法与先前研究中使用的方法进行比较: VinePPO [31], 在线-差分隐私保护 (Online-DPO) [19], (异步) PPO [59], 以及 (近端) RLOO [2] 此外, 本文实现了一种 GRPO [61] 基线 (详情见附录 E.1)。

偏好微调 (PFT) 指标和基线。本文遵循 Noukhovitch 等人的评估设置 [48], 采用在 67 亿“黄金”奖励模型下获胜率作为主要度量指标 [27]。此外, 我们还报告了学习策略与参考策略之间的近似 KL 距离——通过困惑度来近似, 以匹配 Noukhovitch 等人提出的评估方法 [48]。根据 Noukhovitch 等人的做法 [48], 我们将 TBA 与以下方法进行比较: 在线 DPO [19]、(异步)PPO [59] 以及 (近端)RLOO [2]。

红队测试 (RT) 指标和基线。本文遵循 Lee 等人的做法 [36], 测量对受害者模型 1024 个采样提示的攻击成功率。我们还通过计算平均成对余弦距离来衡量这些测试时生成提示的多样性。我们将结果与以下方法进行比较: SFT、PPO [59] (使用 Lee 等人提出的新颖性奖励 [36])、REINFORCE [79, 66]、RLOO [1]、Online DPO [55]、GFlowNet (单演员 TB) [36]。

5.3 TBA 重新定义效率-性能帕累托前沿

我们假设, 通过混合异步强化学习与从非策略数据中学习的轨迹平衡目标, T 中 TBA 可以 (a) 减少资源浪费和训练时间, 并 (b) 提高性能-

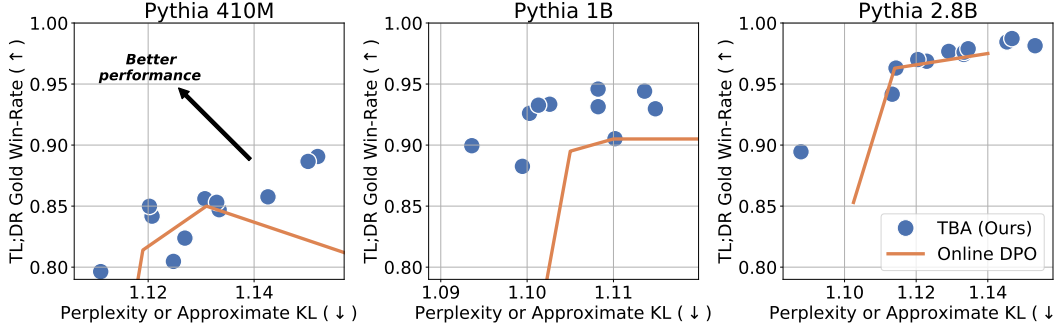


图 4: TBA 在 PFT 摘要任务上定义了一个新的 KL 对比胜率帕累托前沿。基线“在线 DPO”的前沿是通过增加非策略程度，从策略在线 DPO 开始创建的，结果来自 [48]。TBA 的前沿是通过改变训练步骤、搜索器数量和 KL 温度调整计划来创建的，具体细节见附录 B。

表 1: TBA 在 PFT 任务 (Pythia 410M) 中优于离策略和在线策略基线。基线来自 Noukhovitch 等人的研究 [48]。最终块使用 16 步离策略进行基线训练，并将 TBA 的同步周期设置为 10，这意味着平均使用 15 步离策略的数据。

Steps/Sync	Metric	Method			
		Online DPO	(Async) PPO	(Proximal) RLOO	TBA
1 (on-policy)	Perplexity/KL ↓	1.13	1.14	1.13	—
	Win Rate ↑	0.85	0.86	0.82	—
2 (async)	Perplexity/KL ↓	1.13	1.14	1.13	—
	Win Rate ↑	0.85	0.84	0.83	—
≈ 15 (async)	Perplexity/KL ↓	1.12	1.10	1.10	1.13
	Win Rate ↑	0.82	0.76	0.77	0.86

通过扩展生成和摄取多样化的响应来提高性能。为了测试这一点，我们主要考虑计算资源匹配的场景，在这种场景下所有方法都可以访问相同数量的 GPU，尽管我们也评估了 TBA 在资源扩展情况下的表现。即使在我们的计算资源匹配实验中，TBA 也能相对较快地生成响应，因为它在所有可用的 GPU 上运行异步搜索，除了一个 GPU。TBA 的训练速度快是因为它不受在线策略生成的限制，而 TBA 快速生成响应确保了对多样化和之前未见过（尽管是离策略）数据的训练。

当在已建立的强化学习问题上进行测试时，由于其与传统方法的不同之处，TBA 可能会表现不佳：它是异步的，离策略的，依赖于较少见的轨迹平衡目标，并且使用每个查询的许多响应来进行更新。¹ 确实，Noukhovitch 等人 [48] 同时指出，对于大型语言模型而言，异步离策略强化学习可能存在局限性，发现增加离策略程度会损害诸如胜率之类的性能指标，或者加剧策略与参考模型之间的偏差。此外，Hou 等人 [24] 发现，从 4 到 8（或 16）个响应生成扩展时，使用 PPO 几乎没有收益。

我们通过计算 MR（图 1）、PFT（图 3 和图 4

以及表 1）和 RT（图 5 和表 6）的帕累托前沿来研究这个问题。请参阅附录 B 获取实验细节。值得注意的是，TBA 在三个任务的多个模型规模上都产生了位于或超越帕累托前沿的结果，这与我们的假设一致，即 TBA 可以有效地使用离策略数据训练大型语言模型。

TBA 训练的速度有了极大的提升，整个过程完全异步进行，没有训练绑定或生成绑定的过程——唯一的非训练时间是在每 k 步时短暂发生。在计算资源匹配的 MR 实验（图 1）中，TBA 将唯一具有可比性能的方法（VinePPO）的训练速度提高了近 $50\times$ 倍，同时提高了准确性约 1.8 %

¹ 我们通过增加每个查询样本的数量来计算 TBA 损失，相较于类似方法如 RLOO（例如，20 个样本对 4 个样本），以减少 TB 目标估计梯度（方程 5）的方差。我们减少了每批查询的数量，以保持较小的批大小，尽管响应进行了扩展。

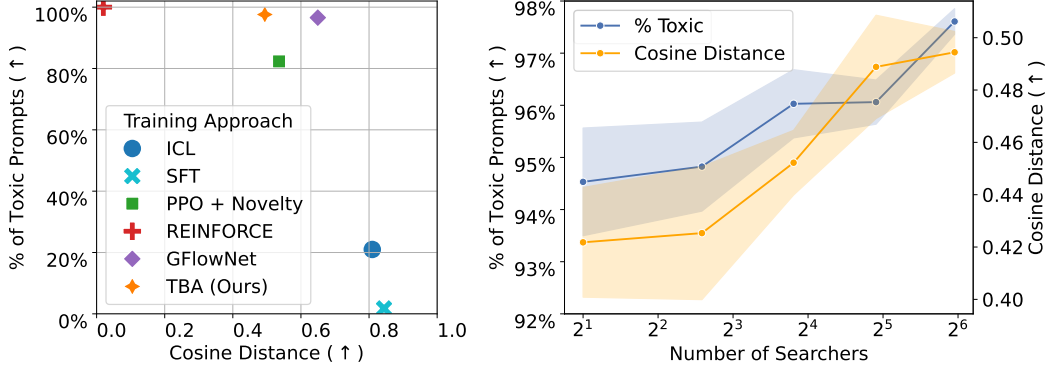


图 5: TBA 达到了 RT 多样性与毒性帕累托前沿, 并随着搜索规模的扩大而改进。(左) 在 Lee 等人的 GPT-2 自动化红队任务中 [36], TBA 在训练时间更少的情况下产生了位于多样性与毒性帕累托前沿的结果。基线数据来自 Lee 等人 [36]。(右) 每个搜索者使用一个 V100 GPU 生成攻击。我们报告了多次运行 GPT-2 自动化红队任务的均值和标准误, 每个搜索器/GPU 数量。

通过 $1.5\times$ 相对于速度优化的异步 DPO 基线 [48] 在计算匹配的 PFT 实验 (图 3) 中, TBA 产生 $\approx 5\times$ 相对于速度优化的异步 DPO 基线的加速 [48], 并且其性能甚至超越了使用在线数据训练的基线模型 (表 1)。在未匹配计算资源的自动化红队实验中 (表 6), TBA 提供 $\approx 7\times$ 与非分布式同步 GFlowNet 基线相比, GPT-2 和 Llama 3.2 1B 的加速效果 [36] 这些结果表明, TBA 是一个有效的并行可扩展搜索框架, 适用于分布式学习, 能够在保持与领先方法竞争力的同时提供显著的速度提升。

5.4 离线策略是否影响性能?

前一节的结果可能令人惊讶, 鉴于已有证据表明离策略性对强化学习训练后的有害影响 [48]。因此, 我们通过研究与离策略性相关的超参数来探讨其影响, 这些超参数我们首先在此进行回顾。分数 m 控制采样尝试近似“在线策略”分布的频率。当 $m = 1$ 时, 训练仅使用最近同步步骤添加的样本, 而 $m = 0$ 则表示选择数据时不考虑其生成时间 (例如, 使用奖励加权)。此外, 参数和缓冲区数据每 k 个训练步骤在搜索器和训练器之间共享: 由于 TBA 并行运行探索和训练, 即使当 $m = 1$ 和 $k = 1$ 时, TBA 也进行离策略训练。具体来说, 以概率 m , TBA 从中央重放缓冲区中选择一个最多离策略更新次数为 $2k - 1$ 的样本。以概率 $1 - m$, TBA 从模型在任何时刻产生的数据中采样, 这可以是离策略的, 其程度取决于训练步数。

为了解增加离策略性对 TBA 的影响, 我们测试了修改 m 对 MR (见图 12) 和 PFT 任务的效果。对于 PFT, 我们在 TL;DR 数据集上训练 Pythia-410M, 并保持其他参数不变, 使用三个值的 m (0.4, 0.5, 0.6)。我们发现胜率波动, 其中 $m = 0.4$ 对应的最低胜率为 0.67, 而 $m = 0.5$ 和 $m = 0.6$ 分别达到了更高的胜率 0.82 和 0.8。这些结果表明, 较高的 m 值通常会导致更高的胜率, 这进一步证实了在线策略更新实际上是最有效的。然而, 对于合理高的 m 值, 包含更多的离策略数据并不会显著降低性能, 在某些情况下甚至可能带来益处。无论如何, 我们对 MR 和 PFT 的研究进一步支持了这样一个观点: 只要将最近的样本纳入其中, 我们可以执行大规模分布式训练, 该训练能够很好地与离策略更新配合。

重要的是, 选择强化学习算法对于有效利用离策略更新至关重要。TBA 是一种完全兼容离策略的算法, 正如图 4 所示, 它显著优于异步在线 DPO, 即使后者处于最接近在线策略的设置。Noukhovitch 等人 [48] 在其实验中确定在线 DPO 是表现最佳的离策略算法 (见表 1), 因此值得注意的是 TBA 改进了这一点。

5.5 通过缩放搜索发现高回报样本

除了在给定计算水平上的改进外，当我们将总计算量进行扩展时，我们也观察到了改进，这显示了 TBA 在大规模分布式强化学习中的潜力。在我们的异步设置中，我们发现增加更多的搜索者可以持续提高攻击成功率和 RT 的多样性（见图 5，右侧）。

这种改进可能源于有更多的搜索者同时探索解决方案空间的不同区域，从而更有效地发现高回报样本。此外，异步更新引入了有益的随机性机会，因此有可能扩大语言组合空间中的搜索覆盖范围。有趣的是，我们还发现了一些证据表明扩展对 PFT 也有帮助（见图 14）。

相关地，在表 6 中，我们展示了通过将最大缓冲区大小扩展到保留更多离策略数据，我们可以用攻击毒性换取更大的攻击多样性。这个 RT 实验使用 Llama 3.2 1B 模型。

6 对更大模型的 TBA 进行扩展

正如第 4 节所述，TBA_l 允许我们测试一个简化的 TBA 版本，并使用更大的模型/上下文进行训练。与 TBA 相比，TBA_l 有较少的参数需要调整：它没有衰减计划对于 β ，PRIME-RL 不从缓冲区采样，而是使用恒定水平的离策略性。一个 TBA_l 配置包括参考策略重置间隔 ρ ，以及一个常数 β 。我们使用 $\rho = 50$ ， $\beta = 0.005$ 。在图 6 中，我们发现了进一步的证据，证明 TBA 可以增强 LLM 的异步强化学习训练，特别是在数据高度离策略的情况下。详见附录 C 和 D 以获取更多详细信息和结果。

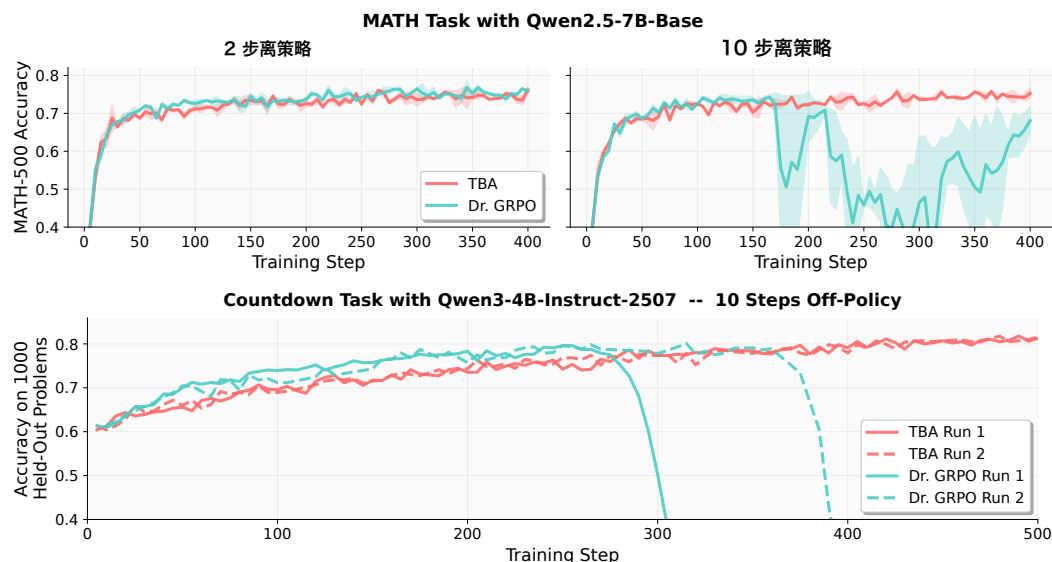


图 6: TBA_l 与 Dr. GRPO 对比。在训练 Qwen 2.5 7B 基础模型时 [70] 使用 MATH 数据集 [20] 或在 Countdown 上训练 Qwen 3 4B 指令模型时 [65] 我们发现 TBA 相对于最先进的方法表现良好，尤其是在高度离策略设置中。详见附录 D 以获取更多结果和细节。

7 讨论

本文介绍了 TBA，一种结合离策略强化学习目标与分布式异步搜索的大语言模型后训练新方法。通过分离搜索节点和训练节点，我们的方法实现了高效的分布式训练并避免了瓶颈，从而在数学推理、自动化红队测试及 RLHF 等后训练任务中取得了显著的性能提升。我们预期这种高度并行且高效的 RL 后训练框架可以扩展到其他有价值的任务中，包括自我改进训练 [82, 25, 17]、基于搜索的推理 [75] 以及新兴的使用 RL 进行推理训练的语言模型范式 [18]。

局限性 轨迹平衡目标在操作轨迹层面时可能会遭受高梯度方差的影响。我们通过增加每个查询的响应采样数量来解决这一问题。未来的工作可以利用学习部分能量函数 [40, 84] 来在策略更新过程中平衡偏差和方差。

更广泛影响 改善大语言模型的 RL 训练策略可以增强其跨领域的实用性，但也伴随着误用、奖励规格不当及意外泛化的风险。随着这些方法的扩展，仔细评估和负责任的部署至关重要。

致谢和资金披露

我们感谢 Michael Noukhovitch 和 Haizhong Zheng 提供的有益反馈。

作者们感谢 CIFAR、NSERC、IVADO 和 Samsung 的资金支持。MJ 获得了 FRQNT 博士奖学金的支持 (<https://doi.org/10.69777/366694>)。

由 LLNL 根据合同 DE-AC52-07NA27344 准备，并得到 LLNL-LDRD 项目 No. 24-ERD-058 (LLNL-CONF-2003261) 的支持。本手稿由 Lawrence Livermore National Security, LLC 根据与美国能源部签订的合同 No. DE-AC52-07NA27344 撰写。美国政府保留权利，出版商在接受文章发表时也承认，美国政府保留非独家、已支付费用、不可撤销、全球范围内的许可，用于发布或复制本手稿的已发布形式，或允许他人出于美国政府目的进行此类操作。

本研究使用了国家能源研究科学计算中心 (NERSC) 的资源，这是一个能源部科学办公室用户设施，使用 NERSC 奖项 ASCR-ERCAP0032802 和 ASCR-ERCAP0032812。

参考文献

- [1] Sina Ahmadi and Aso Mahmudi. Revisiting and amending Central Kurdish data on UniMorph 4.0. In *Proceedings of the 20th SIGMORPHON workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 38–48, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.sigmorphon-1.5. URL <https://aclanthology.org/2023.sigmorphon-1.5>.
- [2] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.
- [3] Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2024.
- [4] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow network based generative models for non-iterative diverse candidate generation. *Neural Information Processing Systems (NeurIPS)*, 2021.
- [5] Yoshua Bengio, Salem Lahlou, Tristan Deleu, Edward J. Hu, Mo Tiwari, and Emmanuel Bengio. GFlowNet foundations. *Journal of Machine Learning Research*, 24(210):1–55, 2023.
- [6] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning (ICML)*, 2023.
- [7] Kianté Brantley, Mingyu Chen, Zhaolin Gao, Jason D Lee, Wen Sun, Wenhao Zhan, and Xuezhou Zhang. Accelerating rl for llm reasoning with optimal advantage regression. *arXiv preprint arXiv:2505.20686*, 2025.
- [8] Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

- [9] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. @articlechristiano2017deep, title=Deep reinforcement learning from human preferences, author=Christiano, Paul F and Leike, Jan and Brown, Tom and Martic, Miljan and Legg, Shane and Amodei, Dario, journal=Advances in neural information processing systems, volume=30, year=2017 . *Advances in neural information processing systems*, 30, 2017.
- [10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [11] Chris Cundy and Adam Gleave. Preference learning with lie detectors can induce honesty or evasion. *arXiv preprint arXiv:2505.13787*, 2025.
- [12] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. Raft: Reward ranked finetuning for generative foundation model alignment. *arXiv preprint arXiv:2304.06767*, 2023.
- [13] Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.
- [14] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023.
- [15] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding code llms in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.
- [16] Google Gemini Team. Gemini: A family of highly capable multimodal models, 2024. URL <https://arxiv.org/abs/2312.11805>.
- [17] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.
- [18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [19] Shangmin Guo, Biao Zhang, Tianlin Liu, Tianqi Liu, Misha Khalman, Felipe Llinares, Alexandre Rame, Thomas Mesnard, Yao Zhao, Bilal Piot, et al. Direct language model alignment from online AI feedback. *arXiv preprint arXiv:2402.04792*, 2024.
- [20] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- [21] Matteo Hessel, Manuel Kroiss, Aidan Clark, Iurii Kemaev, John Quan, Thomas Keck, Fabio Viola, and Hado van Hasselt. Podracer architectures for scalable reinforcement learning. *arXiv preprint arXiv:2104.06272*, 2021.
- [22] Zhang-Wei Hong, Idan Shenfeld, Tsun-Hsuan Wang, Yung-Sung Chuang, Aldo Pareja, James R. Glass, Akash Srivastava, and Pulkit Agrawal. Curiosity-driven red-teaming for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [23] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado Van Hasselt, and David Silver. Distributed prioritized experience replay. *arXiv preprint arXiv:1803.00933*, 2018.
- [24] Zhenyu Hou, Pengfan Du, Yilin Niu, Zhengxiao Du, Aohan Zeng, Xiao Liu, Minlie Huang, Hongning Wang, Jie Tang, and Yuxiao Dong. Does rlhf scale? exploring the impacts from data, model, and method. *arXiv preprint arXiv:2412.06000*, 2024.

- [25] Edward J Hu, Moksh Jain, Eric Elmoznino, Younesse Kaddar, Guillaume Lajoie, Yoshua Bengio, and Nikolay Malkin. Amortizing intractable inference in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [26] Shengyi Huang, Jiayi Weng, Rujikorn Charakorn, Min Lin, Zhongwen Xu, and Santiago Ontañón. Cleanba: A reproducible and efficient distributed reinforcement learning platform. In *International Conference on Learning Representations (ICLR)*, 2023.
- [27] Shengyi Huang, Michael Noukhovitch, Arian Hosseini, Kashif Rasul, Weixun Wang, and Lewis Tunstall. The n+ implementation details of rlhf with ppo: A case study on tl; dr summarization. *arXiv preprint arXiv:2403.17031*, 2024.
- [28] Prime Intellect. Prime-rl, 2025. URL <https://github.com/PrimeIntellect-ai/prime-rl>.
- [29] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [30] Jiaming Ji, Mickel Liu, Josef Dai, Xuehai Pan, Chi Zhang, Ce Bian, Boyuan Chen, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. *Neural Information Processing Systems (NeurIPS)*, 2024.
- [31] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment, 2024. URL <https://arxiv.org/abs/2410.01679>.
- [32] Minsu Kim, Taeyoung Yun, Emmanuel Bengio, Dinghui Zhang, Yoshua Bengio, Sungsoo Ahn, and Jinkyoo Park. Local search GFlowNets. *International Conference on Learning Representations (ICLR)*, 2024.
- [33] Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 REINFORCE samples, get a baseline for free!, 2019. URL <https://openreview.net/forum?id=r1lgTGL5DE>.
- [34] Tomasz Korbak, Ethan Perez, and Christopher L Buckley. Rl with kl penalties is better viewed as bayesian inference. *arXiv preprint arXiv:2205.11275*, 2022.
- [35] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [36] Seanie Lee, Minsu Kim, Lynn Cherif, David Dobre, Juho Lee, Sung Ju Hwang, Kenji Kawaguchi, Gauthier Gidel, Yoshua Bengio, Nikolay Malkin, et al. Learning diverse attacks on large language models for robust red-teaming and safety tuning. *International Conference on Learning Representations (ICLR)*, 2025.
- [37] Zhenghao Lin, Zhibin Gou, Yeyun Gong, Xiao Liu, Yelong Shen, Ruochen Xu, Chen Lin, Yujiu Yang, Jian Jiao, Nan Duan, et al. Rho-1: Not all tokens are what you need. *arXiv preprint arXiv:2404.07965*, 2024.
- [38] Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models. *arXiv preprint arXiv:2505.24864*, 2025.
- [39] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [40] Kanika Madan, Jarrod Rector-Brooks, Maksym Korablyov, Emmanuel Bengio, Moksh Jain, Andrei Nica, Tom Bosc, Yoshua Bengio, and Nikolay Malkin. Learning gflownets from partial episodes for improved convergence and stability, 2023. URL <https://arxiv.org/abs/2209.12782>.

- [41] Dan Malkin, Tomasz Limisiewicz, and Gabriel Stanovsky. A balanced data approach for evaluating cross-lingual transfer: Mapping the linguistic blood bank. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4903–4915, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.naacl-main.361. URL <https://aclanthology.org/2022.naacl-main.361>.
- [42] Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlowNets. *Neural Information Processing Systems (NeurIPS)*, 2022.
- [43] Meta AI Llama Team. The llama 3 herd of models. August 2024. URL <https://arxiv.org/abs/2407.21783>.
- [44] Volodymyr Mnih. Asynchronous methods for deep reinforcement learning. *arXiv preprint arXiv:1602.01783*, 2016.
- [45] Ofir Nachum, Mohammad Norouzi, Kelvin Xu, and Dale Schuurmans. Bridging the gap between value and policy based reinforcement learning. *Advances in neural information processing systems*, 30, 2017.
- [46] Arun Nair, Praveen Srinivasan, Sam Blackwell, Cagdas Alcicek, Rory Fearon, Alessandro De Maria, Vedavyas Panneershelvam, Mustafa Suleyman, Charles Beattie, Stig Petersen, et al. Massively parallel methods for deep reinforcement learning. *arXiv preprint arXiv:1507.04296*, 2015.
- [47] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback, 2021. URL <https://arxiv.org/abs/2112.09332>, 2021.
- [48] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*. OpenReview.net, 2025.
- [49] Nikolas Nüsken and Lorenz Richter. Solving high-dimensional Hamilton–Jacobi–Bellman PDEs using neural networks: perspectives from the theory of controlled diffusions and measures on path space. *Partial Differential Equations and Applications*, 2(4):48, 2021.
- [50] OpenAI. Gpt-4 technical report, 2023. URL <https://openai.com/research/gpt-4>.
- [51] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- [52] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. *International Conference on Learning Representations (ICLR)*, 2022.
- [53] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3419–3448, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.225. URL <https://aclanthology.org/2022.emnlp-main.225>.
- [54] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019.
- [55] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Neural Information Processing Systems (NeurIPS)*, 2023.

- [56] Jarrod Rector-Brooks, Kanika Madan, Moksh Jain, Maksym Korablyov, Cheng-Hao Liu, Sarath Chandar, Nikolay Malkin, and Yoshua Bengio. Thompson sampling for improved exploration in gflowtnets. *arXiv preprint arXiv:2306.17693*, 2023.
- [57] Lorenz Richter, Ayman Boustati, Nikolas Nüsken, Francisco J. R. Ruiz, and Ömer Deniz Akyildiz. Vargrad: A low-variance gradient estimator for variational inference. *Neural Information Processing Systems (NeurIPS)*, 2020.
- [58] Mikayel Samvelyan, Sharath Chandra Raparthy, Andrei Lupu, Eric Hambro, Aram H. Markosyan, Manish Bhatt, Yuning Mao, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, Tim Rocktaschel, and Roberta Raileanu. Rainbow teaming: Open-ended generation of diverse adversarial prompts. *arXiv preprint arXiv:2402.16822*, 2024.
- [59] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [60] Marcin Sendera, Minsu Kim, Sarthak Mittal, Pablo Lemos, Luca Scimeca, Jarrod Rector-Brooks, Alexandre Adam, Yoshua Bengio, and Nikolay Malkin. Improved off-policy training of diffusion samplers. *Neural Information Processing Systems (NeurIPS)*, 2024.
- [61] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [62] Max W Shen, Emmanuel Bengio, Ehsan Hajiramezani, Andreas Loukas, Kyunghyun Cho, and Tommaso Biancalani. Towards understanding and improving GFlowNet training. *International Conference on Machine Learning (ICML)*, 2023.
- [63] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. *Neural Information Processing Systems (NeurIPS)*, 2022.
- [64] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Neural Information Processing Systems (NeurIPS)*, 2020.
- [65] Zafar Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards, 2025. URL <https://arxiv.org/abs/2505.24760>.
- [66] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.
- [67] Yunhao Tang, Zhaohan Daniel Guo, Zeyu Zheng, Daniele Calandriello, Rémi Munos, Mark Rowland, Pierre Harvey Richemond, Michal Valko, Bernardo Ávila Pires, and Bilal Piot. Generalized preference optimization: A unified approach to offline alignment. *arXiv preprint arXiv:2402.05749*, 2024.
- [68] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [69] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- [70] Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL <https://qwenlm.github.io/blog/qwen2.5/>.
- [71] Nikhil Vemgal, Elaine Lau, and Doina Precup. An empirical study of the effectiveness of using a replay buffer on mode discovery in gflowtnets. *arXiv preprint arXiv:2307.07674*, 2023.

- [72] Siddarth Venkatraman, Moksh Jain, Luca Scimeca, Minsu Kim, Marcin Sendera, Mohsin Hasan, Luke Rowe, Sarthak Mittal, Pablo Lemos, Emmanuel Bengio, Alexandre Adam, Jarrid Rector-Brooks, Yoshua Bengio, Glen Berseth, and Nikolay Malkin. Amortizing intractable inference in diffusion models for vision, language, and control. *Neural Information Processing Systems (NeurIPS)*, 2024.
- [73] Bertie Vidgen, Tristan Thrush, Zeerak Waseem, and Douwe Kiela. Learning from the worst: Dynamically generated datasets to improve online hate detection. In *ACL*, 2021.
- [74] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.
- [75] Ziyu Wan, Xidong Feng, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. Alphazero-like tree-search can guide large language model decoding and training. In *International Conference on Machine Learning (ICML)*, 2024.
- [76] Haoxiang Wang, Yong Lin, Wei Xiong, Rui Yang, Shizhe Diao, Shuang Qiu, Han Zhao, and Tong Zhang. Arithmetic control of llms for diverse user preferences: Directional preference alignment with multi-objective rewards. *arXiv preprint arXiv:2402.18571*, 2024.
- [77] Taiyi Wang, Zhihao Wu, Jianheng Liu, Jianye Hao, Jun Wang, and Kun Shao. Distrl: An asynchronous distributed reinforcement learning framework for on-device control agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- [78] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? *Neural Information Processing Systems (NeurIPS)*, 2023.
- [79] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.
- [80] Wei Xiong, Hanze Dong, Chenlu Ye, Ziqi Wang, Han Zhong, Heng Ji, Nan Jiang, and Tong Zhang. Iterative preference learning from human feedback: Bridging theory and practice for rlhf under kl-constraint. In *International Conference on Machine Learning (ICML)*, 2024.
- [81] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [82] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Neural Information Processing Systems (NeurIPS)*, 2022.
- [83] David W Zhang, Corrado Rainone, Markus Peschl, and Roberto Bondesan. Robust scheduling with GFlownets. *International Conference on Learning Representations (ICLR)*, 2023.
- [84] Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. The lessons of developing process reward models in mathematical reasoning. *arXiv preprint arXiv:2501.07301*, 2025.
- [85] Xin Zhao, Yongkang Liu, Kuan Xu, Jia Guo, Zihao Wang, Yan Sun, Xinyu Kong, Qianggang Cao, Liang Jiang, Zujie Wen, Zhiqiang Zhang, and Jun Zhou. Small leak can sink a great ship—boost rl training on moe with icepop!, sep 2025. URL <https://ringtech.notion.site/icepop>. AntGroup Technical Report.
- [86] Haizhong Zheng, Jiawei Zhao, and Bedi Chen. Prosperity before collapse: How far can off-policy rl reach with stale data on llms? *arXiv preprint arXiv:2510.01161*, 2025.
- [87] Brian D Ziebart. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy*. Carnegie Mellon University, 2010.
- [88] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

A TBA Gradient Analysis

Below, we reproduce for convenience the TBA loss $\mathcal{L}_{\text{TB}}^{\text{VarGrad}}$ from Eq. 5, then provide the gradient of the corresponding maximization objective $\mathcal{J}_{\text{TB}}$.

$$\mathcal{L}_{\text{TB}}^{\text{VarGrad}}(\mathbf{B}; \theta) = \frac{1}{BK} \sum_{i=1, j=1}^{i=B, j=K} \left(\text{STOP-GRAD}[\log \hat{Z}(\mathbf{x}^{(i)})] + \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \log \pi_{\text{ref}}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \frac{1}{\beta} r_{\phi}(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)}) \right)^2. \quad (8)$$

We first multiply everything inside the square operation of Eq. 5 by -1 , which is equivalent to multiplying the entire equation by 1 and gives

$$\mathcal{L}_{\text{TB}}^{\text{VarGrad}}(\mathbf{B}; \theta) = \frac{1}{BK} \sum_{i=1, j=1}^{i=B, j=K} \left(\log \pi_{\text{ref}}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) + \frac{1}{\beta} r_{\phi}(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)}) - \log \hat{Z}(\mathbf{x}^{(i)}) \right)^2. \quad (9)$$

The gradient with respect to θ then becomes

$$\nabla \mathcal{L}_{\text{TB}}^{\text{VarGrad}}(\mathbf{B}; \theta) = \frac{1}{BK} \sum_{i=1, j=1}^{i=B, j=K} -2 \left(\log \pi_{\text{ref}}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) + \frac{1}{\beta} r_{\phi}(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)}) - \log \hat{Z}(\mathbf{x}^{(i)}) \right) \nabla \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}), \quad (10)$$

and we can turn this into a maximization problem instead of a minimization problem by multiplying the right side by -1 :

$$\nabla \mathcal{J}_{\text{TB}}^{\text{VarGrad}}(\mathbf{B}; \theta) = \frac{1}{BK} \sum_{i=1, j=1}^{i=B, j=K} 2 \left(\log \pi_{\text{ref}}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) + \frac{1}{\beta} r_{\phi}(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)}) - \log \hat{Z}(\mathbf{x}^{(i)}) \right) \nabla \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}). \quad (11)$$

We multiply the right hand side by $\frac{\beta}{2}$, rescaling the gradient without changing the minimizer. Also, we define $\bar{\text{KL}}^{(i)} = \frac{1}{K} \sum_{j=1}^K \hat{\text{KL}}^{(i,j)} = \frac{1}{K} \sum_{j=1}^K \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}) - \log \pi_{\text{ref}}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)})$ and define $\bar{r}^{(i)} = \frac{1}{K} \sum_{j=1}^K r^{(i,j)} = \frac{1}{K} \sum_{j=1}^K r_{\phi}(\mathbf{y}^{(i,j)}; \mathbf{x}^{(i)})$ to rewrite Eq. 4 as $\log \hat{Z}(\mathbf{x}^{(i)}) = \frac{1}{\beta} \bar{r}^{(i)} - \bar{\text{KL}}^{(i)}$:

$$\nabla \mathcal{J}_{\text{TB}}(\mathbf{B}; \theta) = \frac{1}{BK} \sum_{i=1, j=1}^{i=B, j=K} A^{(i,j)} \nabla \log \pi_{\theta}(\mathbf{y}^{(i,j)} | \mathbf{x}^{(i)}), \quad (12)$$

where $A^{(i,j)} = (r^{(i,j)} - \bar{r}^{(i)}) - \beta(\hat{\text{KL}}^{(i,j)} - \bar{\text{KL}}^{(i)})$. In sum, TBA appears to update model weights using a REINFORCE algorithm with a mean-reward baseline [79] and a KL-divergence-regularized reward [88]. However, note that $\hat{\text{KL}}^{(i,j)}$ is only an estimate of the KL divergence in the on-policy case because off-policy data is not sampled from π_{θ} by definition, causing loss of equivalence to mean-baseline REINFORCE with a KL-regularized reward in the asynchronous settings that TBA operates in. Deviating further, TBA may also reset the reference policy similar to Liu et al. [38].

附录 B TBA 实验细节

正如第 4 节和第 5.4 节所讨论的，TBA 引入了新的超参数。最值得注意的是，这些包括（1）同步周期 k ，即两次连续模型-缓冲区同步之间的训练步骤数；以及（2）最符合策略的概率 m ，即采样最符合策略的数据的概率——即，在最近一次同步期间添加到缓冲区中的样本。在列出超参数值时，我们明确了那些特定于 TBA 的超参数，并包含了对文本中讨论/可视化内容的引用。

对于 MR 和 PFT，我们通过构建 Hugging Face TRL 库中的 RLOO 训练器类来实现 TBA ([74])。对于 MR 和 PFT，我们分别对基线超参数进行了修改，如表 2 和表 3 所示。

我们的 TBA 实时实现基于李等人使用的 TB 训练器 [36]，我们主要使用其代码的默认超参数，差异详见附录 B.3。

B.1 GSM8K 数学推理 (MR)

所有基线和 TBA 结果均采用相同的起点，RhoMath-1B [37] 由 Kazemnejad 等人在 GSM8K 训练数据上微调的模型 [31] – “realtreetune/rho-1b-sft-GSM8K”在 Hugging Face 上。基线模型达到了 40.3 % 测试准确率。对于 VinePPO 基线，训练时间是根据他们报告的每步训练 380 秒和 GSM8K 训练 650 步来估算的 [31] 我们通过 Hugging Face TRL 训练器实现 GRPO 基线，具体细节见附录 E.1。

表 2: GSM8K MR 任务的 TBA 训练超参数。

Hyperparameter	Value	Reference
Model	Rho-1B SFT on GSM8K	
Learning Rate	1×10^{-5}	
Learning Rate Schedule	Warmup Stable Decay	
Learning Rate Warmup Steps	50	
Learning Rate Stable Steps	450	
Learning Rate Decay Steps	500	
Generation Temperature	0.7	
Max Prompt Token Length	512	
Response Length	512	
Number of Prompts per Batch	7	
Number of Completions per Prompt	20	K in Section 4.1
Batch Size (effective)	140	
Number of Training Steps	1000	
Total Prompts Seen	7000	
Total Episodes	140000	
<i>TBA-specific hyperparameters</i>		
Beta (KL coefficient) Initial Value	0.012	β in Equation 5
Beta Final Value	0.004	β in Equation 5
Beta Linear Decay Schedule End Step	500	β in Equation 5
Number of Samples per Prompt	24	S in Section 4.1
Most-On-Policy Sampling Probability	0.95	m in Section 5.4
Sync Period	2	k in Section 5.4, Figure 2
Number of Searchers	3	Searchers in Figure 2
Number of Trainers	1	Trainer in Figure 2
Reward-Based Sampling Prioritization	Uniform	Section 4.2
Initial Completions in Buffer	500	Buffer at Step 0 in Figure 2

在图 1 中，TBA 使用表 2 中的设置并进行了少量修改，以将训练时间进一步缩短 30%，降至 4xA100 GPU 上的 82 分钟。特别地，在初步测试中（见附录 E），我们发现使用表 2 中的超参数在最后 300 步没有带来性能提升。因此，我们将训练时长从 1000 步减少到 700 步（98000 个回合，批大小为 140）。此外，我们还进行了以下修改，试图降低缩短运行的方差：350 步稳定的学习率（从 450 步减少），0.014 初始 Beta 值

表 3: TL;DR PFT 任务的 TBA 训练超参数。对于 PFT 任务，我们加速了 Beta 的衰减。具体而言，Beta 线性衰减计划结束步骤设置为训练步骤数的一半，但在训练步骤数的八分之一处突然终止衰减并将 Beta 设置为其最终值（例如，对于 256 步，第 32 步）。这相当于用 KL/困惑度换取胜率，而我们发现 TBA 设置下的 KL/困惑度相对较低。

Hyperparameter	Value	Reference
Model	Pythia SFTed on TL;DR	
Learning Rate	3×10^{-6}	
Learning Rate Schedule	Linear	
Generation Temperature	0.7	
Max Token Length	1024	
Max Prompt Token Length	512	
Response Length	128	
Number of Prompts per Batch	8	
Number of Completions per Prompt	20	K in Section 4.1
Batch Size (effective)	160	
Number of Training Steps	256	
Total Prompts Seen	2048	
Total Episodes	40960	
<i>TBA-specific hyperparameters</i>		
Beta (KL coefficient) Initial Value	1	β in Equation 5
Beta Final Value	0.05	β in Equation 5
Beta Linear Decay Schedule End Step	See caption	β in Equation 5
Number of Samples per Prompt	20	S in Section 4.1
Most-On-Policy Sampling Probability	0.5	m in Section 5.4
Sync Period	10	k in Section 5.4, Figure 2
Number of Searchers	3	Searchers in Figure 2
Number of Trainers	1	Trainer in Figure 2
Reward-Based Sampling Prioritization	Softmax of Score	Section 4.2
Initial Completions in Buffer	10000	Buffer at Step 0 in Figure 2

(从 0.012 增加到 0.014)。我们进行了三次实验——获得的性能分别为 55.8 %，53.9 %，和 54.1 %——并在图 1 中报告了平均准确率 54.6 %。

局限性和未来工作 我们在图 1 中展示的 700 步结果的标准误差为 0.6 %，这比我们在原始 1000 步设置中观察到的方差略大（图 12 左下角图中的蓝色线显示了 1000 步运行的均值和标准误差）。未来工作可以进一步探索缩短运行时间的方差缩减方法/超参数（以及更广泛的 TBA/RL 领域）。

处理方差的一种方法是基于保留的验证集选择检查点，这种方法用于生成 VinePPO 的结果 [31]。本文并未采用此方法，但注意到我们的结果可能会因此显著提升（ $\approx 1\%$ ）。具体而言，我们的每次运行在训练过程中的某个时刻通常会产生比最终步骤更高的性能——这可以预期，因为模型性能会在其收敛到训练结束时的平均值附近波动（例如，请再次参见图 12 左下角图中的蓝色曲线）。尽管它低于训练过程中任何时刻达到的最大性能，但我们报告了最终步骤的性能，这是 TBA 用户在不使用诸如提前停止等技术的情况下可以期望获得的性能。

B.2 简言之，偏好微调 (PFT)

对于图 3 中的 PFT，我们采用了表 3 中的设置以及一个持续时间更长的运行，共进行了 425 次更新。

对于图 4 中的 PFT（性能前沿测试），我们通过修改表 3 所示的训练步骤、搜索器数量和 beta 线性衰减计划来创建 TBA（训练边界调整）前沿。我们分别训练 256、**425**、625、725 和 825 步，并使用 2、3、**4** 和 7 个搜索器节点进行搜索。在改变搜索器数量时，我们没有观察到性能有显著的模式，但我们发现 KL/困惑度值和胜率倾向于更高。

更多的训练步骤（这是进一步优化策略的预期权衡）。我们注意到 28 亿参数模型在初始运行中没有产生广泛的 KL/困惑度值，因此我们在该模型规模下也进行了运行（825 步，分别使用 2 个和 4 个搜索者），采用了较慢的 beta 线性衰减计划（在第 140 步而不是第 104 步达到最终的 beta 值）。这种计划变化的效果是降低了 KL/困惑度和胜率（另一个预期的权衡）。

对于表 1 中的 PFT 实验，我们采用了表 3 中的设置，除了训练步数为 625 步（100000 个回合），因为我们发现增加更多的步数可以提高胜率而不显著增加困惑度（近似 KL）。此外，本次实验仅使用 2 个搜索者。图 14 展示了步数和搜索者数量的影响。

局限性和未来工作 我们的所有 PFT 结果均在 32 位精确度下运行，并未使用 DeepSpeed，而基线对比实验使用了该加速包 [48]。更低的精确度和类似的训练加速包可能会进一步提高我们展示的速度提升。相关地，对于我们的 2.8 B 实验，我们使用梯度检查点来将更多样本放入微批次中，这导致了在这一规模下的减速（即，在这种情况下，我们仅比基线有 3.8 倍的速度提升）。我们将框架与适当加速包的优化留作未来工作。最后，我们在缓冲区中使用了大量（10000）初始完成样本，未来的工作应确认较小的数量（例如 1000）是否同样有效——请注意，较小的数量对 GSM8K 效果良好。

B.3 自动化红队测试 (RT)

与我们的 MR 和 PFT 实现不同，我们的 RT 实现采用了 Lee 等人的代码 [36] 作为基线，因此不遵循 Hugging Face TRL 训练器的风格。我们在其轨迹平衡训练器方法的背景下讨论超参数。我们基本上采用了他们的代码，唯一的区别在于我们的 TBA 实现使用的回放缓冲区比李等人更大。[36] 以适应我们对轨迹的缩放搜索。此外，与李等人不同的是，[36] 当缓冲区满时，我们移除最旧的样本而非奖励值最低的样本。

对于表 6 中的 Llama 实验结果，我们的超参数选择主要遵循 Lee 等人的设置 [36]。我们训练 5000 步，批大小为 128。对于温度采样，我们分别使用 0.7 和 2.0 作为低温和高温。我们使用 1000 步的奖励计划时间范围。语言模型计划结束时间为 1.2，其计划的时间范围为 2000。我们基于奖励优先进行采样。我们使用 Beta 0.05。我们使用同步周期 (k) 10。我们使用 6 个搜索者。我们使用最接近策略概率 (m) 0.5 和 0.6，分别结合最大缓冲区大小 150000 和 130000。通过拥有较小的最大缓冲区大小和较大的 m ，后一种设置预计会更关注最近生成的数据，并优先考虑奖励而非多样性，这在表 6 中得到了验证。

对于图 5 和表 6 中的 GPT2 结果，我们的超参数选择再次主要遵循 Lee 等人的设置 [36]。我们使用批大小 128 训练 5000 步。我们使用 Beta 0.05。我们使用同步周期 (k) 10。我们使用最接近策略概率 (m) 0.5。我们在所有实验中将最大缓冲区大小限制为 100000；我们还防止缓冲区在第 4000 步后继续增长，以鼓励关注更近期的数据（更大的最接近策略概率 m 可能会产生类似的效果）。我们测试搜索者数量分别为 2、6、14、30、62，并对每种配置使用 3 次运行以计算误差条和平均值。

局限性和未来工作 在表 6 中，随着搜索者数量的增加，出现了减速现象。我们认为这主要是通过我们代码的新版本得到了解决，该版本采用了更高效的缓冲区通信，但我们尚未重新运行这些结果来确认这一点。无论如何，鉴于 TBA 能够有效利用大规模数据生成的能力，开发更高效的 TBA 实现是一个有趣的方向，值得未来研究。

C TBA' Experiment Details

我们的 TBA' 实验训练 Qwen 2.5 7B 基础模型 [70] 在 MATH 数据集上 [20] 或者训练 Qwen 3 4B 指令模型 2507 [81] 在 9000 个生成的倒计时问题上 [65] 分别在 MATH-500 或 1000 个预留的倒计时问题上进行测试。

在图 6 中，我们将 TBA' 与 PRIME-RL[28] 中的 Dr. GRPO 实现 [39] 进行了比较。两种方法的超参数相同，具体数值见表 4。请注意，当 $\rho = 1$ （并且没有异步性）或当 $\beta = 0$ 时，TBA' 与 Dr. GRPO 相同。关于 TBA' 和消融研究的更多细节，请参阅附录 D。

表 4: MATH-500 和 Countdown 任务的训练超参数。对于 Countdown 任务，我们训练最多 500 步而不是 400 步。此外，Countdown 实验使用的是 Qwen 3 4B Instruct 2507 而非 Qwen 2.5 7B Base。

Hyperparameter	Value	Reference
Model	Qwen 2.5 7B Base	
Learning Rate	1×10^{-6}	
Generation Temperature	1.0	
Max Prompt Sequence Length	1024	
Max Model Length	3072	
Number of Prompts per Batch	32	
Number of Completions per Prompt	16	K in Section 4.1
Batch Size (effective)	512	
Number of Training Steps	400	
Total Prompts Seen	12800	
Total Episodes	204800	
CISPO IS lower/upper bound	0/8	
<i>TBA'-specific hyperparameters</i>		
Beta (KL coefficient)	0.005	β in Equation 5
Reference policy reset period	50	ρ in Section 6

D TBA' 的消融研究和在强化学习领域的上下文文化

TBA' 通过添加重要性采样 (IS) 并采用 CISPO 样式的 IS 比率裁剪来扩展 TBA[8]。如图 7 所示，某些任务可以从 IS 中受益，尽管它并非对所有任务都至关重要。TBA' 还在训练过程中重置参考策略而不是减少 β 。图 8 显示，重置参考策略对于获得高性能至关重要，其效果类似于 IS。

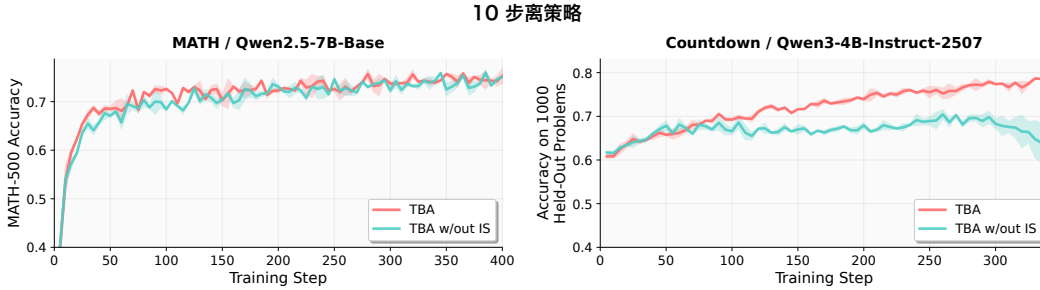


图 7: 由于 IS 在某些任务上的有用性，我们在 TBA' 中添加了裁剪的重要性采样。

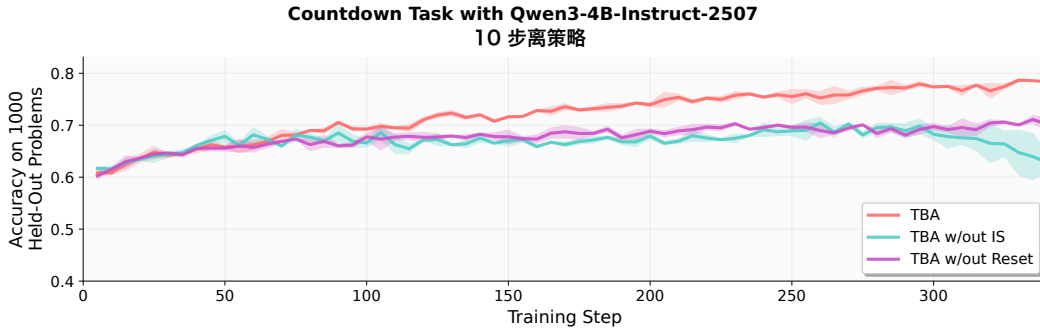


图 8: 我们展示了关闭重置以及关闭裁剪重要性采样的效果在 TBA' 中。

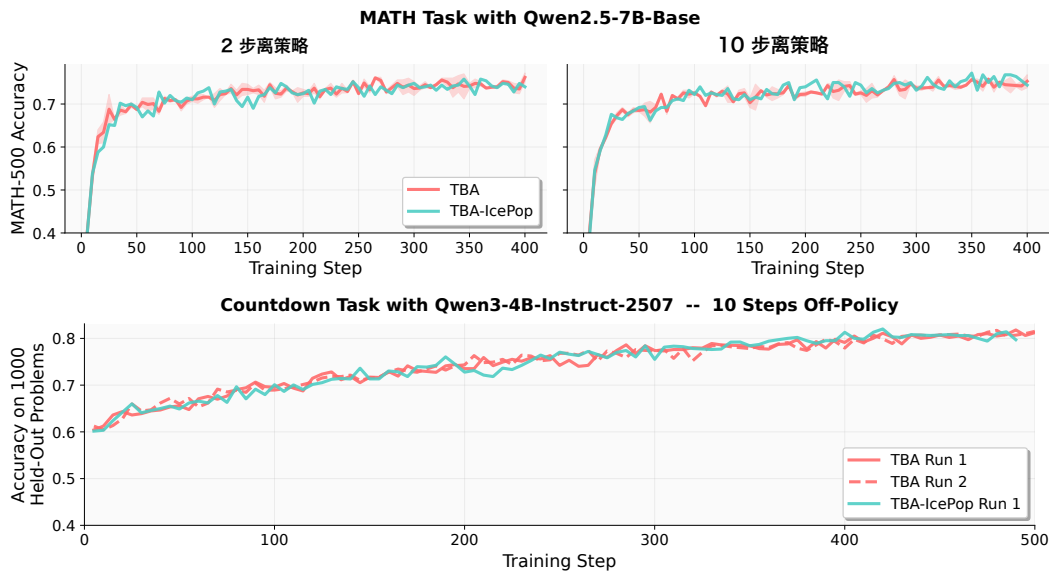


图 9：当用 IcePop 掩码替换重要性采样裁剪时，TBA 仍然有效。

值得注意的是，过于频繁地重置参考策略可能会消除 KL 正则化的益处。这一重置周期 ρ 因此是 TBA 的关键超参数，（沿同 β ：两者取较大值 ρ 和 β 预计将增加向原始参考策略的优化压力，而较小的值则允许任务奖励的自由优化，但可能风险稳定性。我们强调，我们的 Dr. GRPO 基线正是 TBA，当 $\beta = 0$ ，该基线在处理离策略数据时显示出较低的鲁棒性（见图 6）。

与裁剪 IS 比率不同，最近的研究表明，对于具有大或小比率的标记，屏蔽梯度可以是有效的 [85, 86]。在图 9 中，我们展示了 TBA 方法进行 KL 正则化（每 50 步重置一次，将对数概率比率添加到奖励中）在使用这种屏蔽（具体是 IcePop 的）而不是 CISPO 样式的裁剪时是有效的。关键在于，如果没有 TBA 的 KL 正则化方法，仅使用这种屏蔽不足以获得对高度离策略数据的鲁棒性，如图 10 所示。

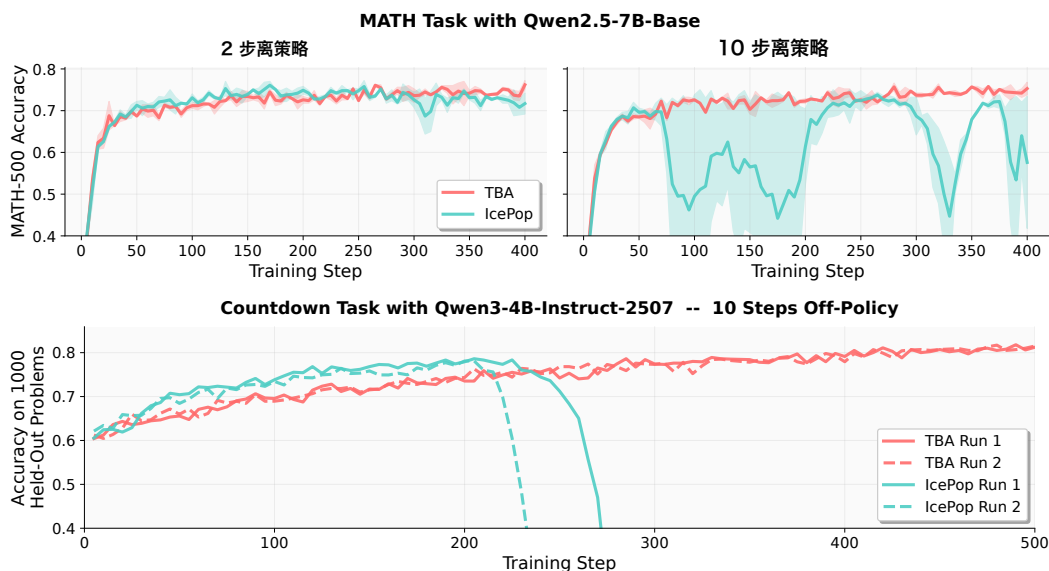


图 10：单独的 IcePop 掩码不足以达到 TBA 的离策略鲁棒性。

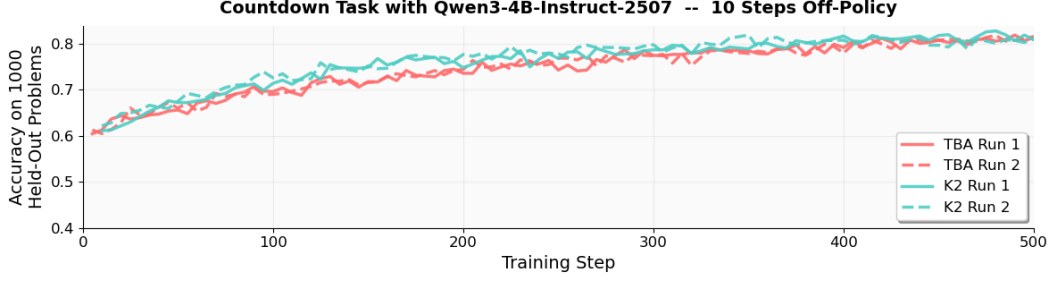


图 11: Kimi K2 进行 KL 正则化的方法类似于 TBA , 并提供了类似的离策略性能。经过一个小规模的超参数研究后, 我们使用 $\tau = 0.005$ 用于 K2。

我们所知最接近 TBA 的方法是 Kimi K1.5 和 K2 [69, 68]。正如 Team 等人在 [68] 中所示, Kimi K2 的对数概率比利用了训练策略和推理策略的概率: 这类似于 KL 正则化, 每一步都将参考策略重置为推理时使用的策略。值得注意的是, 如果更接近原始参考策略是有益的, 那么由于频繁的重置, 这种方法可能会表现不佳。而 K2 方法无法改变重置周期, 也无法使用与推理策略不同的参考策略, 相比之下, TBA 具有使用较长或较短重置周期的灵活性。在我们的实验中, TBA 仅每 $\rho = 50$ 步将参考策略重置为当前训练策略, 从而可能提供更强的对基础策略的遵循。另一个主要区别在于, TBA 基于第 3.2 节给出的原则性推导, 在控制变量中包含了平均对数概率比。图 11 显示, Kimi 方法和 TBA 具有相似的离线策略性能。未来的工作可以研究在需要严格遵循基础策略的情况下 (例如, 诱导奖励黑客行为的设置, 如 Cundy 和 Gleave 在 [11] 中所述) 性能差异。

梯度比较 我们考虑的目标变体的梯度如下。请注意, 我们在所有方法中保持剪辑策略不变, 测试关键方法方面 (用红色标出) 对离线策略性能的影响。相关地, 在所有情况下, 我们在求和标记级梯度时使用 PRIME-RL 默认的 GRPO 样式的归一化。

$$\nabla \mathcal{J}_{\text{CISPO}}(\theta) = \sum_{j=1}^K \sum_{t=1}^{|y^{(j)}|} \text{sg} \left(\min(\lambda_t^{(j)}, 8) \frac{r^{(j)} - \bar{r}}{\text{std}(\{r^{(j)}\}_{j=1}^K)} \right) \nabla \log \pi_{\theta}(y_t^{(j)} | \mathbf{x}, y_{i < t}^{(j)}), \quad (13)$$

$$\nabla \mathcal{J}_{\text{Dr. GRPO}}(\theta) = \sum_{j=1}^K \sum_{t=1}^{|y^{(j)}|} \text{sg} \left(\min(\lambda_t^{(j)}, 8) (r^{(j)} - \bar{r}) \right) \nabla \log \pi_{\theta}(y_t^{(j)} | \mathbf{x}, y_{i < t}^{(j)}), \quad (14)$$

$$\nabla \mathcal{J}_{\text{Kimi-K2}}(\theta) = \sum_{j=1}^K \sum_{t=1}^{|y^{(j)}|} \text{sg} \left(\min(\lambda_t^{(j)}, 8) (r^{(j)} - \bar{r} - \tau \log \lambda^{(j)}) \right) \nabla \log \pi_{\theta}(y_t^{(j)} | \mathbf{x}, y_{i < t}^{(j)}), \quad (15)$$

$$\nabla \mathcal{J}_{\text{TBA}}(\theta) = \sum_{j=1}^K \sum_{t=1}^{|y^{(j)}|} \text{sg} \left(\min(\lambda_t^{(j)}, 8) (r^{(j)} - \bar{r} - \beta (\log \Lambda^{(j)} - \overline{\log \Lambda})) \right) \nabla \log \pi_{\theta}(y_t^{(j)} | \mathbf{x}, y_{i < t}^{(j)}), \quad (16)$$

其中 $\lambda_t^{(j)} = \frac{\pi_{\theta}(y_t | \mathbf{x}, y_{i < t}^{(j)})}{\pi_{\text{ref}}(y_t | \mathbf{x}, y_{i < t}^{(j)})}$ 是 K 序列上的均值, $\log \Lambda^{(j)} = \frac{1}{|y^{(j)}|} \sum_{t=1}^{|y^{(j)}|} \log \lambda_t^{(j)}$ 是 $\log \lambda_t^{(j)}$ 的均值, $\overline{\log \Lambda} = \frac{1}{K} \sum_{j=1}^K \log \Lambda^{(j)}$ 是 $\log \Lambda^{(j)}$ 的均值。最后, 请注意, TBA 每 ρ 步将 π_{ref} 重置为 π_{θ} 。

Brantley 等人在 [7] 中也使用了一个类似 TBA 的回归目标, 但相反, 他们离线学习一个扮演控制变量角色的价值函数。

E TBA GSM8K 消融研究

我们在图 1 中采用了基于一系列围绕表 2 所示超参数进行的试验实验得出的 GSM8K 结果的超参数。在图 12 中，我们展示了改变我们认为的关键超参数的效果。关于 TBA 超参数对 GSM8K 的影响，我们注意到以下观察结果。

1. 与 PFT 不同，对于 GSM8K MR 任务而言， m 适度较大较为重要（图 12，左上）。同样地，更频繁的同步也是有益的（图 12，右上）。这些结果共同表明，GSM8K 的性能对离策略性比其他任务（如 PFT）更为敏感。
2. 我们发现，WSD 调度可以增加稳定性（图 12，右下）。
3. 使用较小的 Beta 最终值倾向于提高性能（图 12，左下），但在接近 0.003 时训练变得不稳定。这表明 GSM8K 的稳定性和准确率之间存在一个由 KL 系数 β 调节的权衡。
4. 初始测试中，批大小为 140 并未提供明显优于更大批大小的结果，但较小的批大小允许更快的训练步骤，这促使我们选择 140 作为批大小。

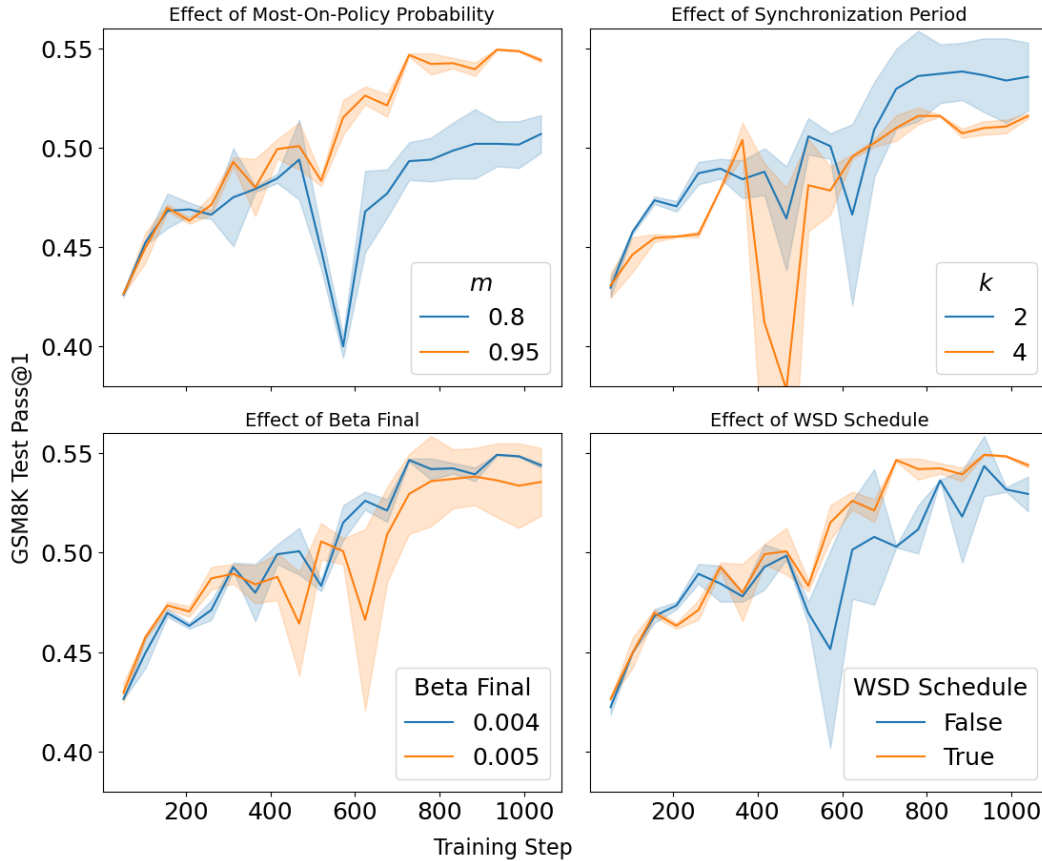


图 12: GSM8K 消融研究。所有实验均从表 2 列出的基础超参数开始，并进行所示修改，除了在右上角图中研究同步周期 k 的情况（此处我们使用 Beta 最终值 0.005，因为 0.004 导致了不稳定，与 $k = 4$ 相关）。我们报告了每种配置两次运行的平均值和标准误。

随后，我们考虑将每个提示的完成次数 K 从 20 增加到 40。在表 5 中，我们可以看到这减少了方差并提高了性能。关键的是，这一结果并不需要更长的训练时间，因为我们只需减少每批中独特的提示数量，以保持批大小恒定。

表 5: 在 GSM8K 上, 将 K 翻倍可提高 TBA 的准确率和方差。

K	Test Accuracy	Standard Deviation
20	54.61	0.85
40	54.89	0.49

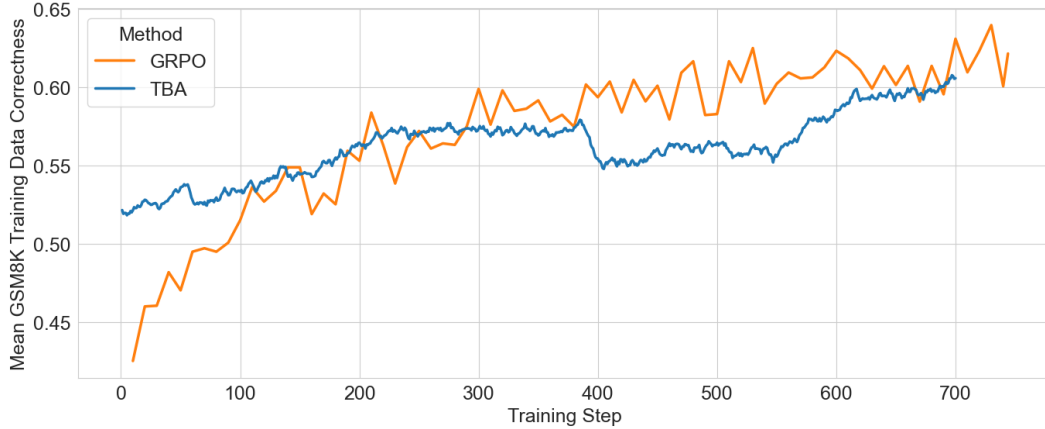


图 13: GRPO 和 TBA 训练动态。TBA 性能通过指数移动平均进行了大量平滑处理（未经平滑处理时, TBA 曲线波动较大, 使得趋势难以观察）, 而 GRPO 性能未进行平滑处理。

E.1 GRPO 基线创建及训练动态比较

我们使用 Hugging Face TRL（版本 0.17.0）实现 GRPO 基线。相对于基线 GRPOConfig 超参数, 我们仅更改以下设置: 训练轮次数量=8, 梯度累积步数=5, 以及每设备训练批大小=32。为了加速训练速度, 我们使用 DeepSpeed 并设置 ZeRO 阶段=2。我们发现这些设置在相同的 4xA100（80GB）节点上能够有效利用 GPU 内存和计算资源, 这与我们在 TBA 实验中使用的节点相同。此外, 批大小（640）、总训练轮次（8）和优化步骤数量（744）基于用于 VinePPO 的 RL 设置 [31]——该论文创建了我们在 GSM8K 实验中应用 GRPO 的 SFT 基线（参见图 1）。

为了得到图 1 中的 GRPO 结果, 我们探索了几种同步 GRPO 的实现变体。有趣的是, 我们也尝试了异步 GRPO, 通过将我们在异步 TBA 代码中使用的损失函数替换为 GRPO 的损失函数来实现。我们在 TBA 的异步设置内测试了几种异步 GRPO 的超参数设置——提高 KL 系数、降低学习率和减少离策略性——但没有找到提供稳定学习的设置。

图 13 展示了最终（同步）GRPO 实现和随机 TBA 复现的平均训练数据得分（GSM8K 正确性）随训练步骤的变化情况。此 GRPO 运行的最终模型用于计算图 1 所示的测试性能, 即 49.05%。此次 TBA 运行达到了 54.51% 的测试准确率, 尽管训练性能水平相似, 仍提高了 5%。

F TBA TL;DR 消融研究

通过 TBA，我们在每小时训练时间中采取的优化步骤远多于竞争方法。因此，我们试图理解增加步数或使用更多搜索器（以达到与竞争方法相同的计算量）如何影响性能。如图 14 所示，胜率随着这些变量的变化而提高，尽管当过度优化策略时（在步数缩放的情况下），困惑度会如预期般恶化。尚不清楚为什么增加搜索器数量的效果类似于步数缩放。然而，存在一种明显的机制，即搜索器缩放可能产生影响：使用更多的搜索器应降低在离线策略采样时选择之前已见过的训练提示的概率（因为缩放搜索器的数量也增加了缓冲区中添加的独特提示的数量）。² 然而，这种效果大小较小且不一致，表明需要进一步研究才能确认这种搜索器缩放趋势（在 PFT 背景下）。

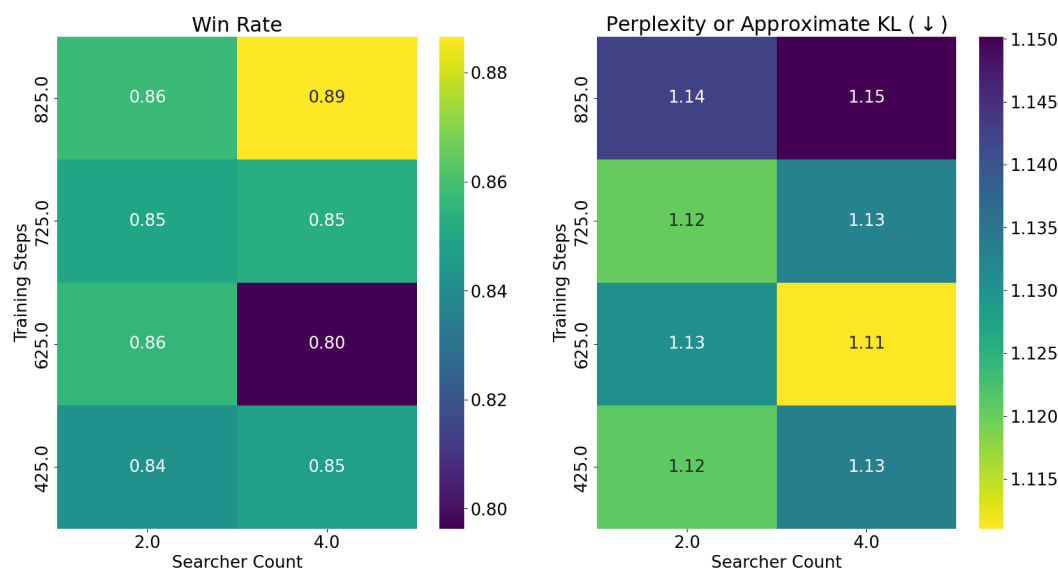


图 14: TL;DR 消融研究。所有实验均从表 3 列出的基本超参数开始，然后进行所展示的修改。增加搜索器节点和训练步数通常会提高胜率，但代价是更高的困惑度。

值得注意的是，在实时（RT）情况下，搜索缩放的效果不同。在这种情况下，只有一个提示，增加搜索器使得找到并训练更高奖励样本的可能性更大。而在 PFT 中，增加搜索意味着每秒有更多的提示生成完成，但任何给定的提示不会因为增加搜索器数量而有更高的奖励样本——这是一个实现选择，可以更改以获得与 RT 中搜索器缩放相同的效果。

G 红队测试附加结果

表 6: TBA 加快了到达红队任务帕累托前沿所需的实时时钟时间。 GFlowNet 的性能数据来自 Lee 等人 [36]，而训练速度由我们使用他们的代码计算得出。对于 GPT-2 模型，TBA 性能随搜索器数量增加而提高。对于 Llama 模型，我们将 TBA 缓冲区的最大容量从 130,000（倒数第二行）扩展到 150,000 样本（最后一行），以换取攻击多样性的增加，同时保留更多的离线策略数据。

Attacker / Victim Model	Training Method	Hardware	Time (h)	Speedup	Cosine Distance	% Toxic Prompts
GPT-2 / GPT-2 + SFT	GFlowNet - Sync	1×V100	11.9	1x	0.65	96.6
	TBA (Ours)	4×V100	1.7	7x	0.42	94.5
	TBA (Ours)	16×V100	2.5	4.8x	0.45	96
	TBA (Ours)	64×V100	2.9	4.1x	0.49	97.6
Llama 3.2 1B / Llama 3.1 8B - Instruct	GFlowNet - Sync	1×A100	37.4	1x	0.32	100.0
	TBA (Ours)	8×A100	5.7	6.6x	0.35	98.1
	TBA (Ours)	8×A100	5.7	6.6x	0.37	94.8